

NCSA Feature Inventory

NCSA — Complete Feature Inventory

AI-Driven Multi-Vendor Network & Firewall Compliance Auditor

Generated 9 September 2026 from the working tree at `E:\NCSA`. Every number in this document was read out of the code or the live API, not estimated. 382 tests pass.

0. Where the website files are

You asked directly, so this comes first.

What	Path	Served at
Home / landing page	<code>E:\NCSA\ncsa\api\static\landing.html</code> (278 lines)	<code>http://127.0.0.1:8000/</code>
Audit console — the page you upload the config file to	<code>E:\NCSA\ncsa\api\static\index.html</code> (178 lines)	<code>http://127.0.0.1:8000/app</code>
Console behaviour	<code>E:\NCSA\ncsa\api\static\app.js</code> (405 lines)	—
Console styling	<code>E:\NCSA\ncsa\api\static\app.css</code>	—
Landing behaviour / styling	<code>landing.js</code> , <code>landing.css</code>	—
3-D firewall visual	<code>appliance.js</code> , <code>graphics.js</code>	—
Separate Next.js marketing site	<code>E:\NCSA\web\</code> (source), <code>E:\NCSA\web\out\index.html</code> (static export)	<code>http://localhost:3000</code>

Both are served by FastAPI from `ncsa/api/app.py` lines 48–62 via `FileResponse`. There are **two** front-ends in the repo. Deciding which one is the demo is an open question — see §6.

1. What the engine actually is

A configuration file goes in; a compliance assessment comes out, with the exact line of the exact file behind every claim.

The design commitment that shapes everything else: **absence is never reported as a positive result**. A control we could not evaluate says so. It never becomes a PASS, and it never becomes a silent zero.

Fact	Value
Security Baseline Model fields	119 , across 18 domains
Compliance rules	81 (6 critical, 26 high, 36 medium, 13 low)
Vendor mapping packs	11
Platforms parsed	10
Platforms with a policy object graph	4
Framework controls indexed	5,785 across 8 catalogues
Python modules	89 files, 16,513 lines
Tests	382 , all passing

Observation states (what we saw)

OBSERVED · DEFAULT_ASSUMED · NOT_OBSERVED · UNPARSED

Result states (what it means)

PASS · FAIL · PARTIAL · NOT_APPLICABLE · UNKNOWN · MANUAL_REVIEW · ERROR

Seven result states, not two. UNKNOWN is first-class: it is the difference between "we checked and it is fine" and "we could not check".

Honest coverage

score_pct is computed over **decided controls only** and is always published beside assessed_pct . A device where we could read 40% of the settings and all of them passed scores 100% on 40% coverage — never 100% full stop.

2. Complete capability matrix

Your original table had two states. The truth now needs four columns, because "reachable from the API" and "has a button in the console" are different things, and collapsing them would overstate what a demo can show.

#	Capability	Built	Tests	API endpoint	Console UI
1	Multi-vendor parsing (6 grammar families)	11 packs	35	POST /assess	✓ Yes
2	Compliance evaluation (81 rules)	308 lines	35	GET /assessment/{id}	✓ Yes
3	Rule hygiene — dead / shadowed / over-broad	322 lines	14	GET .../hygiene	✓ Yes
4	Remediation generation	202 lines	✓	GET .../remediation	✓ Yes
5	Training queue + approval loop	477 lines	12	GET .../training, POST /training/approve	✓ Yes
6	Risk scoring	156 lines	✓	inside /assessment/{id}	✓ Internal
7	Framework provenance (8 catalogues)	1,082 lines	✓	GET /frameworks	✓ Yes
8	Reachability — would this traffic pass?	289 lines	15	POST .../reach	⚠ API only
9	Multi-device topology	393 lines	8	POST /topology, GET .../interfaces	⚠ API only
10	Change tracking / snapshots / diff	344 lines	15	POST .../snapshot, GET .../diff	⚠ API only
11	Recertification workflow	232 lines	6	GET .../recertification	⚠ API only
12	Log correlation	199 lines	5	POST .../logs	⚠ API only
13	Parser cross-check (consensus)	358 lines	10 + 9	GET .../consensus	⚠ API only
14	Host firewall audit (Windows / iptables)	329 lines	14	POST /hostfw/iptables, POST /hostfw/local	⚠ API only
15	Golden corpus / regression gate	187 lines	6	internal	✗ CI only
16	Capability boundary self-report	—	✓	GET /health	⚠ API only

Nothing in this table is unreachable any more. In the version of the table you pasted, seven capabilities were built, tested, and callable from nowhere. All seven now have endpoints. What remains is that eight of them have **no button in the console** — the work in §5.

The 22 live API routes

GET	/	landing page
GET	/app	audit console
POST	/assess	upload one or many configs
GET	/assessments	everything assessed this session
GET	/assessment/{id}	findings, coverage, evidence
GET	/assessment/{id}/hygiene	dead / shadowed / over-broad rules
GET	/assessment/{id}/remediation	fix commands, lockout-checked
GET	/assessment/{id}/training	unrecognised settings queue
POST	/training/approve	approve a learned mapping
POST	/assessment/{id}/reach	would this traffic be permitted?
POST	/topology	build a fabric from several devices
GET	/assessment/{id}/interfaces	the addressing topology infers from
POST	/assessment/{id}/snapshot	record state for later comparison
GET	/assessment/{id}/diff	what changed since last snapshot
GET	/assessment/{id}/recertification	rules due for review / deletion
POST	/assessment/{id}/logs	correlate syslog against rules
GET	/assessment/{id}/consensus	independent-parser agreement
POST	/hostfw/iptables	audit an uploaded iptables ruleset
POST	/hostfw/local	audit the API host's own firewall
GET	/frameworks	catalogue sizes + licence note
GET	/platforms	supported platforms
GET	/health	liveness + capability boundary

3. Feature detail

3.1 Parsing — packs are data, never code

Six grammar-family readers cover every vendor syntax we support: `indented` · `braces` · `fortinet_block` · `sonicos_exp` · `xml` · `json`

A new vendor is a **YAML file**, not a code change. That is the whole extension mechanism.

Pack	Platform	Validated against
<code>cisco.yaml</code>	Cisco IOS-XE	real config
<code>cisco_asa.yaml</code>	Cisco ASA	real config
<code>sonicwall.yaml</code> , <code>sonicwall_exp.yaml</code>	SonicOS	real NSA 3700 export (2.7 MB)
<code>juniper.yaml</code> , <code>juniper_xml.yaml</code>	Junos SRX	real config
<code>fortinet.yaml</code>	FortiOS	constructed fixture
<code>paloalto.yaml</code>	PAN-OS	real running-config (86 KB)
<code>arista.yaml</code>	Arista EOS	constructed fixture — version 0.9
<code>aruba.yaml</code>	HPE Aruba AOS-CX	constructed fixture — version 0.9
<code>aws.yaml</code>	AWS security groups	real export

Packs validated only against constructed fixtures carry version `0.9` and a tripwire test that fails if anyone marks them 1.0 without checking against real device output. **Real configs find bugs constructed fixtures cannot** — the PAN-OS download exposed an XML reader bug within minutes of being used.

3.2 Policy object graph — 4 platforms

Rules become vendor-neutral `SecurityRule` objects, and every downstream analysis works on them without knowing the vendor.

Platform	Builder	Lines	Evidence
SonicOS	<code>sonicos_builder.py</code>	207	real NSA 3700 export
Junos / SRX	<code>junos_builder.py</code>	176	real config
PAN-OS	<code>panos_builder.py</code>	381	real running-config
FortiOS	<code>fortios_builder.py</code>	412	constructed fixture
Windows / iptables	<code>hostfw_builder.py</code>	129	live collection

The PAN-OS and FortiOS builders were added most recently. Four vendor semantics they handle that a naive port-oriented reading gets backwards:

1. **App-ID is PAN-OS's primary match criterion, not the port.** A rule permitting application `4shared` on `application-default` ports does not permit a port. These rules are marked undecidable for port questions and disclosed, rather than reported as permitting traffic they may not.
2. **`application-default` is not `any`.** Treating an unrecognised service token as unconstrained reads a tightly scoped rule as wide open.
3. **An absent flag means opposite things on different vendors.** On FortiOS and PAN-OS an absent `status / disabled` means **enabled**; on SonicOS an absent enabled flag means **off**. Carrying one assumption across would silently disable a live rulebase and report a permissive firewall as safe.
4. **Negation inverts the match.** `set srcaddr-negate enable` means everything *except* the listed addresses. Read literally it means the exact opposite of what the device does, so it is marked unevaluable instead.

3.3 Reachability

Answers "would this traffic be permitted?" with first-match-wins evaluation, **naming the rule that decided it** so a reviewer can check the verdict rather than take it.

Three honesty properties:

- A rule above the match that could not be evaluated is **disclosed as a caveat**, because a match there would have decided differently.
- `permitted` is tri-state — `True` / `False` / `None`. Undecidable is never flattened into denied.
- If the deciding rule is scoped to zones the query did not name, the answer says the zone was **assumed**. On FortiOS and PAN-OS every rule is zone-scoped, so this affects nearly every answer.

3.4 Rule hygiene

Detects: `unused_rule` · `disabled_rule` · `shadowed_rule` · `redundant_rule` · `overly_permissive` · `orphaned_objects` · `disputed_shadow`

Unevaluable rules are reported **beside** the findings. A rule we could not resolve is not a clean rule, and a summary that hid them would report a policy as tidier than we can confirm. On the real SonicWall: 329 rules examined, 150 fully resolved, 180 unevaluable, 375 findings — all four numbers published.

`disputed_shadow` is unusual and deliberate: when our analysis calls a rule unreachable but the device counted matches against it, we report **our own analysis as wrong** rather than dropping the contradiction.

3.5 Change tracking

Snapshots a device, then reports what changed. The key distinction: **device change and analysis change are reported separately**. A new pack version altering a verdict is not configuration drift, and conflating the two would have an operator chasing a change nobody made.

Identity is matched on strong identifiers (serial), not filename — a rename is not a new device, and a shared filename is not one device.

3.6 Recertification

Which firewall rules need a human decision, and which are safe to delete. Deletion requires **three independent signals agreeing**: no traffic, no owner, and no security purpose still needed. Any one alone is a bad reason to delete a firewall rule. On the real SonicWall: 267 rules due for review, 227 deletion candidates.

3.7 Log correlation

Answers the question rule hygiene cannot: *is this rule genuinely unused, or was its counter reset?* Syslog evidence can **refute a zero counter**, and traffic with no matching log entries is itself a finding. Unparsed log lines are counted, never discarded.

3.8 Parser cross-check (consensus)

Runs independent third-party parsers alongside our own and reports agreement. Two sources agreeing **promotes a caveat to a conclusion**; disagreement is surfaced rather than resolved by picking a favourite.

3.9 Host firewall audit

Windows Firewall and iptables rules become the same `SecurityRule` objects, so the appliance hygiene analyser runs on a laptop with **no host-specific analysis code**. Two host semantics are preserved rather than smoothed away:

- Windows Firewall has **no evaluation order** (block beats allow regardless of position), so shadow analysis is suppressed — a "shadowed rule" finding there would describe semantics the platform does not have.
- A profile whose default action was never observed is reported as unobserved, not assumed to block.

`POST /hostfw/local` sits behind an explicit `?enable=true` opt-in because, unlike every other endpoint, it reads the server rather than an upload.

3.10 Training loop

Unrecognised settings are queued, classified, and offered for approval. An approved mapping must pass the **golden corpus regression gate** before it is accepted — a learned mapping that breaks a known-good assessment is rejected.

3.11 Framework provenance

Catalogue	Controls	Licence handling
NIST SP 800-53	1,196	public domain — full text
DISA STIG	553	public domain — full text
CIS Benchmarks	3,506	identifier only — text never leaves the machine
ISO/IEC 27001:2022	121	identifier + short title only
NIST SP 800-171 r3	130	public domain — full text
PCI DSS 4.0	279	identifier only
CMMC	0	unpopulated, with a stated reason
NERC CIP	0	unpopulated, with a stated reason

Licence enforcement is in the **type system** (`License.PUBLIC_DOMAIN` vs `IDENTIFIER_ONLY`), not in a policy document — copyrighted text cannot be emitted by construction.

The zero rows are important. CMMC Level 2 is 800-171 **Rev 2** (110 requirements); we hold Rev 3 (130). Deriving one from the other would be wrong, so the catalogue is empty and `unpopulated()` explains why. A revision guard rejects a Rev 3 file placed in the Rev 2 slot.

Provenance chains: **STIG** → **CCI** → **NIST 800-53** → **ISO 27001**, and → **800-171**.

3.12 Capability self-report

`GET /health` publishes which analyses this build can actually perform, read from the code at runtime. The claim and the code cannot drift apart, and a caller sees the boundary before hitting it.

4. What makes this different

1. **Honest coverage.** Score over decided controls only, always beside assessed percentage.
 2. **Absence is never a pass.** Seven result states; `UNKNOWN` is first-class.
 3. **Evidence per finding.** File and line for every claim. A test asserts that no value is ever claimed without evidence.
 4. **Evidenced absence.** "Telnet is off" cites all three allow-lists checked, not silence.
 5. **Refusals name the boundary.** An analysis that cannot run says which platforms *can* run it, and marks itself `not_a_finding`.
 6. **Unevaluable rules are disclosed**, not silently skipped.
 7. **Vendor semantics are respected**, including where they contradict each other (absent-flag polarity, unordered platforms, App-ID matching).
 8. **Packs are data.** A new vendor is a YAML file.
 9. **Unvalidated packs are marked 0.9** with a tripwire test.
 10. **Licence compliance in the type system.**
 11. **Regression-gated learning.** No mapping is approved that breaks the golden corpus.
 12. **We report our own analysis as wrong** when the device contradicts it (`disputed_shadow`).
-

5. The gap — and how to close it

Eight capabilities are built, tested, and callable from the API, but have **no console view**. This is now the single largest gap between what NCSA does and what a demo can show.

Each needs a UI panel; none needs new engine work.

Priority	Capability	Console view to build	Effort
1	Reachability	Query form (src / dst / port / zones) → verdict card showing the deciding rule + caveats	Small
2	Rule hygiene detail	Exists, but does not show the unevaluable list	Small
3	Change tracking	"Snapshot" button + a diff timeline with device-change vs analysis-change split	Medium
4	Recertification	Table of rules due, owner, expiry, deletion candidates with the 3 signals shown	Medium
5	Topology	Multi-select assessments → fabric diagram + path query, with the "adjacency is inferred" caveat on every answer	Large
6	Log correlation	Syslog upload → per-rule traffic evidence	Medium
7	Consensus	Parser agreement panel with disagreements highlighted	Small
8	Host firewall	"Audit this machine" tab	Small

Also outstanding, engine-side:

- Distinguish *"no mapping exists for this field"* from *"mapped, but the config is silent"* — both currently surface as UNKNOWN.
- Validate the Arista, Aruba and FortiOS packs/builders against **real captured configs** and lift them from 0.9 to 1.0.
- `load_packs` silently swallows malformed packs (`except: continue`) — a broken pack should be loud.
- Graph builders for Cisco ASA and IOS-XE (would take graph analyses from 4 platforms to 6).
- API-based config collection (SSH/REST pull) — not built at all.

6. Website guidelines

6.1 The two-front-end problem — decide this first

There are two front-ends in the repo:

	FastAPI console (:8000)	Next.js site (:3000)
Location	<code>nlsa/api/static/</code>	<code>web/</code>
Talks to the engine	Yes, directly	No — static export
Has the 3-D firewall visual	Yes	Yes
Can run an audit	Yes	No

Recommendation: make the FastAPI console the product, and fold the Next.js site's design language into it. It is the only one that can actually run an audit, and a marketing site that cannot demo the tool is a liability during judging. The alternative — pointing the Next.js site at the API — is more work and duplicates the console you already have.

6.2 Recommended multi-page structure

Page 1 — Home / landing

- Hero: what NCSA does, in one sentence a judge understands in 4 seconds
- The 3-D firewall visual (already built, [appliance.js](#))
- Four proof numbers: 10 platforms · 81 rules · 5,785 framework controls · 382 tests
- Primary call to action: **Run an audit** → the audit page
- Vendor logo strip (shipped vs roadmap, honestly separated)

Page 2 — Audit console *(the page you upload the config to)*

- See §6.3, this is the important one

Page 3 — Capabilities

- The 16-row matrix from §2, rendered as cards
- Honest status badges — do not paint API-only capabilities as shipped UI

Page 4 — Frameworks & provenance

- The 8 catalogues with control counts
- The provenance chain STIG → CCI → NIST → ISO / 800-171 as a diagram
- The licence note stated plainly — it is a differentiator, not a limitation

Page 5 — How it works

- Pipeline: fingerprint → pack → reader → SBM → rules → findings
- The 6 grammar families
- "A new vendor is a YAML file" — show an actual pack excerpt

Page 6 — Why it is different

- The 12 points from §4, with the honest-coverage explainer front and centre

Page 7 — Docs / API

- Link to the live OpenAPI docs at [/docs](#)
- The 22-route table

6.3 The audit page — detailed specification

This is the page that wins or loses the demo. Structure it in four zones:

Zone 1 — Upload

- Large drop target: "Drop a firewall or switch configuration here"
- Accepts multiple files (bulk ingestion is a deliverable — show it)
- Show detected vendor/platform **immediately** on drop, before assessing — this proves the fingerprinter works
- A redaction toggle, defaulted **on**, with the reason stated: redaction removes addressing, which makes topology inference impossible
- Sample-config buttons so a judge with no config file can still run it

Zone 2 — Result header

- Score **and** coverage side by side, never the score alone

- Explicit caption: "87% of 64 decided controls · 71% of the device assessed"
- Device identity: vendor, model, OS version, hostname, serial
- Counts by result state — all seven, including UNKNOWN and NOT_APPLICABLE

Zone 3 — Findings

- Grouped by severity, filterable by framework
- **Every finding expands to show the exact file line** — this is the single most persuasive thing in the product; make it one click, not two
- Framework provenance chips on each finding (STIG / NIST / CIS ID)
- Remediation command shown inline, with the lockout warning where one applies

Zone 4 — Deeper analysis tabs

- Rule hygiene · Reachability · Recertification · Change · Topology · Consensus
- Each tab that has no data for this platform must say **why**, using the `analysis_ran: false` / `supported_platforms` fields the API already returns
- Never render an empty chart for an analysis that did not run — that is the exact failure mode the engine was built to avoid, and it would undo the product's main claim in the UI layer

6.4 Design rules

- Keep the existing colour scheme and the glassmorphism treatment
 - The UNKNOWN pill must use the muted colour, **never** the pass colour — there is already a test asserting this
 - Every number on screen should be traceable to an endpoint; no decorative statistics
 - Motion should serve comprehension (pipeline flow, packet path), not decorate
 - Responsive: tables scroll inside their own container, the page body never scrolls horizontally
-

7. Test coverage by area

Area	Tests
Pipeline end-to-end	35
API routes	33
FortiOS graph builder	24
Schema / SBM	23
RAG / knowledge	20
Topology (8) · Recertification (6) · Log correlation (5)	19
NLP signals	18
Universal detectors	16
PAN-OS graph builder	15
Change tracking + reachability	15
Rule hygiene	14
Host firewall	14
Approval / training gate	12
XML reader	11
Parser cross-check	10
Palo Alto pack	10
Show-output reader	10
Third-party parser adapter	10
Consensus reconciliation	9
Fortinet pack	8
Arista + Aruba packs	16 (8 parametrised × 2)
Golden corpus	6
Total	382

NCSA — deliverable due 5 September 2026. Document generated from the working tree; every figure is read from code or the live API.